

Omi-Lisp: Emergence from `Atoms.compareExchange`

Status: Canonical

Scope: How Lisp-like structure emerges from the single atomic primitive

Foundation: `bind`, `apply`, `eval`, `digest`, the `iff`

Part I — The Primitive

§ 1. The One Operation

...

`Atoms.compareExchange(array, index, expected, replacement)`

...

This is the entire protocol. Everything else is emergent.

The operation:

- Reads the value at `array[index]`
- Compares it with `expected`
- If equal, writes `replacement`
- Returns the previous value

All in one uninterruptible step.

§ 2. The Three Phases

The primitive contains three phases:

...

`bind` — the conditional: (`expected` $\rightarrow$ `replacement`)

`apply` — the invocation: the compare-and-swap

`eval` — the return: the previous value

...

All three happen together. There is no gap. The physical primitive is the logical primitive.

Three phases, one operation.

§ 3. The Fourth Primitive

...

`digest` — read, consider, print

...

The `digest` reads the state, computes the significance (via popcount and XOR), and writes the result. It closes the loop.

§ 4. The Base Equivalence

...

`position(n)  $\Leftrightarrow$  period(n-1, n, n+1)`

...

The position holds iff the period holds. This is the 2!.

The iff is the base.

Part II — The Emergence of Lisp

§ 5. Why Lisp Emerges

Lisp emerges because the primitive has the shape of Lisp's core operations:

Lisp OMI-Lisp Role

car read the expected the head

cdr read the remainder the tail

cons bind construct a pair

eval eval evaluate

atom the guard the atomicity

nil NULL:U unbound, unresolved

t Bind:K bound, resolved

lambda the knot function descriptor

quote the trace entry immutable record

cond the horn clause conditional derivation

Every Lisp primitive has an atomic analog.

§ 6. Why the Emergence Is Natural

Because:

- Code is data — the swap list is both program and structure
- Uniform representation — everything is swaps
- Structural evaluation — no value computation, only structure
- Homoiconicity — the program and its representation are the same

Lisp's properties emerge from the primitive's properties.

The primitive is Lisp-shaped.

Part III — The Knot

§ 7. The Knot as the Fundamental Type

bind(a, b) produces a knot — a bidirectional relation between two items.

```
...js
class Knot {
  constructor(a, b) {
    this.a = a;
    this.b = b;
  }

  get(key) {
    if (key === this.a) return this.b;
    if (key === this.b) return this.a;
    return undefined;
  }
}
...
```

The knot is:

- Symmetric: bind(a, b) = bind(b, a)
- Reversible: both directions are held
- Non-destructive: arguments are unchanged
- Structural: records a relation, computes nothing

The knot is the Lisp cons cell, but symmetric.

§ 8. Why Symmetric

In traditional Lisp:

```
...lisp
(cons a b) → (a . b)
(car (a . b)) → a
(cdr (a . b)) → b
...
```

Order matters. car and cdr return different things.

In OMI-Lisp:

```
...lisp
(bind a b) → {a ↔ b}
(knot.get a) → b
(knot.get b) → a
...
```

Order doesn't matter. Both directions hold.

Symmetric cons is what enables sharing.

§ 9. Why This Matters

Because two peers can bind their coordinates in either order and get the same knot. In traditional Lisp, order matters, so two peers binding in different orders get different pairs.

Symmetric cons is the sharing primitive.

Part IV — The Three Operations

§ 10. bind

```
...
bind : (item, item) → knot
...
```

bind constructs a relation. It doesn't compute a value. It doesn't transform its arguments. It records a relation.

bind is the constructor.

§ 11. apply

```
...
apply : (knot, args) → result
...
```

apply invokes a knot as a function. The knot acts as a function descriptor.

apply is the invoker.

§ 12. eval

```
...
eval : (knot) → value
...
```

eval extracts a value from a knot. The knot acts as a value descriptor.

eval is the extractor.

§ 13. The Categorical Structure

...

bind — Monad — composition

apply — Functor — lifting

eval — Comonad — extraction

...

Monad + Functor + Comonad. This is the minimum categorical toolkit for a computation.

Three primitives, three categorical structures.

Part V — The Digest

§ 14. The Fourth Primitive

...

digest : (ruler) → value

...

The digest reads the ruler, computes the significance, and writes the result.

```js

function digest(ruler) {

// Read

const reading = read(ruler);

// Consider

const significance = popcount(reading) ^ xor(reading);

// Print

write(significance);

// Loop

return digest(ruler);

}

...

The digest is the loop.

### § 15. The F-Mean

The digest computes the generalized F-mean of the ruler:

...

$M_p(x_1, \dots, x_n) = ((1/n) \sum x_i^p)^{1/p}$

...

The mean order  $p$  is determined by the observer's position.

The digest is the F-mean operation.

### § 16. The Digest as the iff

The digest is the iff in operation:

...

read  $\Leftrightarrow$  print

...

The read and the print are equivalent. Neither exists without the other.

The digest is the iff.

---

## Part VI — The Ruler

### § 17. The Eight Slots

...

ruler[0]  $\rightarrow$  diagonal (the origin)  
ruler[1]  $\rightarrow$  size (the unit count)  
ruler[2]  $\rightarrow$  top  
ruler[3]  $\rightarrow$  bottom  
ruler[4]  $\rightarrow$  right  
ruler[5]  $\rightarrow$  left  
ruler[6]  $\rightarrow$  forward  
ruler[7]  $\rightarrow$  backward

...

### § 18. The Two Groups

...

2!  $\rightarrow$  indices 0, 1  $\rightarrow$  the frame  
3!  $\rightarrow$  indices 2..7  $\rightarrow$  the operations

...

The ruler is  $2! + 3! = 8$  slots long.

### § 19. The Orthogonality

Three levels:

- Internal — axes within a group
- External — groups to each other
- Parent — child to container

The 2! and 3! are separate orthogonal groups.

Two groups, one ruler.

---

## Part VII — The Lisp Emergence

### § 20. The Core Forms

From the primitive, the Lisp's core forms emerge:

...

(cons a b)  $\rightarrow$  bind(a, b)  
(car (cons a b))  $\rightarrow$  a  
(cdr (cons a b))  $\rightarrow$  b  
(eval (cons a b))  $\rightarrow$  the value at the knot

...

### § 21. The Composite Forms

From composition:

```
...
(lambda (x) ...) → a knot with a function body
(let ((x v)) ...) → a nested binding
(if c t e) → a guarded swap
(cond ...) → a Horn clause
(quote x) → a trace entry
...
```

## § 22. The Macro Forms

From the primitive's symmetry:

```
...
(defmacro ...) → a bind that produces binds
(quote x) → an immutable trace entry
(eval x) → the extraction of the trace's value
...
```

Every Lisp form emerges from bind, apply, eval, digest.

---

## Part VIII — The OMI-Lisp Notation

### § 23. The Original Notation

The original OMI-Lisp used:

```
...
omi---imo (the origin boundary)
o---o/--/?---@---@ (the receipt grammar)
(a . b) (dot notation)
omi-0100-03bf-... (the one-long-word)
...
```

### § 24. The Translated Notation

The translated notation uses standard web primitives:

```
...
HTTP/1.1 headers → the citation carrier
WebVTT cues → the timeline
Regex constraints → the validation grammar
DOM geometry → the projection surface
MediaStreams → the substrate
...
```

### § 25. The Dot Notation Replacement

The original dot:

```
```lisp  
(car . cdr)  
(gpio12 . high)  
(user.intent . open-door)  
...
```

Becomes the <dt>/<dd> pair:

```
```html
```

```

<dt id="car" data-omi-pair="cdr">CAR</dt>
<dd data-omi-pair="car">CDR</dd>

<dt id="gpio12" data-omi-pair="high">GPIO12</dt>
<dd data-omi-pair="gpio12">HIGH</dd>

<dt id="user.intent" data-omi-pair="open-door">Intent</dt>
<dd data-omi-pair="user.intent">Open Door</dd>
...

```

The dot is the <dt>/<dd> pair.

---

## Part IX — The Range Selector

### § 26. The -4D to -1D Pipeline

```

...
0x00..0x3F → the OMI-list
...

```

Within the range:

```

...
0x00..0x0F → -4D → control tetrahedra
0x10..0x1F → -3D → separator hierarchy (FS/GS/RS/US)
0x20..0x2F → -2D → delimiter envelope
0x30..0x3F → -1D → alphanumeric sieve
...

```

### § 27. Why 64 Values

Because:

```

...
64 = 26
 = the first 64 ASCII characters
 = the address range of the OMI-list
...

```

The 0x00..0x3F range is exactly the OMI-list's address space.

### § 28. The Regex Constraints

```

...js
const G = Object.freeze({
 // -4D: control tetrahedra
 CONTROL_TETRA: /^[x00-x0F]$/,

 // -3D: separator hierarchy
 FILE_SEP: /^[x1C]$/,
 GROUP_SEP: /^[x1D]$/,
 RECORD_SEP: /^[x1E]$/,
 UNIT_SEP: /^[x1F]$/,

 // -2D: delimiter envelope
 DELIMITER: /^[x20-x2F]$/,

```

```
// -1D: alphanumeric sieve
ALPHANUMERIC: /^[x30-x3F]$/

// Composed patterns
MNEMONIC: /^(d\d)([A-Z_]?[a-z_]+)(d\d):\32\1$/
PALINDROME: /^(d\d)[A-Za-z_\-](d\d):\2[0-9_\-]\1$/
));
...
```

The regex set is the OMI-list's grammar.

---

## Part X — The Processing Pipeline

### § 29. The Six Steps

```
...

recognize → raw input arrives
cite → OMI parses and identifies
validate → Tetragrammatron tests
record → receipt written to ring
project → Metatron renders
inspect → user reads
...
```

### § 30. The Boundary Rules

```
...

1. No recognition makes something true.
2. No notation makes something true.
3. No citation makes something true.
4. Only validation accepts.
5. Only receipt records.
6. Only Metatron projects.
7. No projection makes something true.
...
```

Only validation accepts.

### § 31. The Lazy Evaluation

```
...

(hardware.port . command) → declaration only
 requires validation
 then hardware projection
...
```

No side effects before receipt.

---

## Part XI — The Four Authorities

### § 32. OMI (Citation)

```
...

Question: "What is being referred to?"
Owns: address parsing, hashing, nibble CPU
Does not: validation, geometry, files
...
```

### § 33. Tetragrammatron (Validation)

...

Question: "Does this fit the accepted rule?"

Owns: 5040 ring, slot5040, Fano incidence, chirality

Does not: citation, geometry, transport

...

### § 34. Metatron (Projection)

...

Question: "How should this be displayed?"

Owns: shape database, geometry, renderers

Does not: citation, validation, transport

...

### § 35. IMO (Carrier)

...

Question: "How is this transported?"

Owns: file I/O, HTTP, S-parse, persistence

Does not: citation, validation, projection

...

### § 36. The Receipt as Shared Invariant

...

Receipt is:

cited by OMI

validated by Tetragrammatron

projected by Metatron

carried by IMO

Receipt is the same across all authorities.

...

Four authorities, one receipt.

---

## Part XII — The Canonical Form

### § 37. The Lisp's Shape

OMI-Lisp is:

...

An Atomic Compare-and-Exchange Lisp

whose primitive is `Atomics.compareExchange`

whose core type is the symmetric knot

whose operations are `bind`, `apply`, `eval`, `digest`

whose base is the iff ( $\text{position} \Leftrightarrow \text{period}$ )

whose ruler is 8 slots ( $2! + 3!$ )

whose pipeline is -4D to -1D

whose authorities are OMI, Tetragrammatron, Metatron, IMO

whose carrier is HTTP/1.1 + TextTracks + MediaStreams

whose validation is regex + Horn clauses

whose projection is DOM geometry

...

## § 38. The Emergence

The Lisp emerges from the primitive because:

- car/cdr emerge from the read phases of compareExchange
- cons emerges from bind
- eval emerges from the return phase
- lambda emerges from the knot's function descriptor
- quote emerges from the trace entry
- cond emerges from the Horn clause
- let emerges from nested bindings

Every Lisp form is a composition of the primitive's phases.

## § 39. The Loop

The loop emerges from the digest:

...  
read → read the ruler  
consider → compute the significance  
print → write the result  
loop → repeat  
...

The loop closes the Lisp.

---

## Part XIII — The Translatable Notation

### § 40. The Original Notation

...  
omi---imo → the origin boundary  
o---o/---/?---@---@ → the receipt grammar  
(a . b) → the dot notation  
omi-0100-03bf-... → the one-long-word  
...

### § 41. The Translated Notation

...  
HTTP/1.1 headers → the citation carrier  
WebVTT cues → the timeline  
Regex constraints → the validation grammar  
DOM geometry → the projection surface  
MediaStreams → the substrate  
...

### § 42. The Correspondence

Original Translated

omi---imo HTTP header with X-Omi-List-Version  
o---o/---/?---@---@ X-VTT-Cue-\* header  
(a . b) <dt>/<dd> pair with data-omi-pair  
one-long-word data-\* attributes on DOM elements  
receipt ring DOM description list + trace log

Every original form has a translated form.

### § 43. The -4D to -1D Pipeline

The original OMI-Lisp's -4D to -1D pipeline becomes the range selector:

```
...
0x00..0x0F → -4D → control tetrahedra
0x10..0x1F → -3D → separator hierarchy
0x20..0x2F → -2D → delimiter envelope
0x30..0x3F → -1D → alphanumeric sieve
...
```

The pipeline is the range selector.

---

## Part XIV — The Canonical Example

### § 44. A Complete Event

```
```html  
<!-- The HTTP wire -->  
<link rel="preload" href="/omi-list.vtt" as="track">  
  
<!-- The OMI-list container -->  
<dl id="omi-list">  
  <dt id="range-0x00" data-omi-layer="-4D" data-omi-range="0x00-0x0F">  
    Control Tetrahedra  
  </dt>  
  <dd data-omi-pair="range-0x00">  
    The 16 control states  
  </dd>  
  
  <dt id="range-0x10" data-omi-layer="-3D" data-omi-range="0x10-0x1F">  
    Separator Hierarchy  
  </dt>  
  <dd data-omi-pair="range-0x10">  
    FS/GS/RS/US  
  </dd>  
  
  <dt id="range-0x20" data-omi-layer="-2D" data-omi-range="0x20-0x2F">  
    Delimiter Envelope  
  </dt>  
  <dd data-omi-pair="range-0x20">  
    Non-alphanumeric delimiters  
  </dd>  
  
  <dt id="range-0x30" data-omi-layer="-1D" data-omi-range="0x30-0x3F">  
    Alphanumeric Sieve  
  </dt>  
  <dd data-omi-pair="range-0x30">  
    Token classes  
  </dd>  
</dl>  
  
<!-- The interactive hit zones -->  
<map name="omi-hitzones">  
  <area id="hit-range-0x00" shape="poly" coords="0,0,0,0" href="#range-0x00">  
  <area id="hit-range-0x10" shape="poly" coords="0,0,0,0" href="#range-0x10">  
  <area id="hit-range-0x20" shape="poly" coords="0,0,0,0" href="#range-0x20">
```

```

    <area id="hit-range-0x30" shape="poly" coords="0,0,0,0" href="#range-0x30">
  </map>

```

```

<!-- The media surfaces -->
<video id="omi-video" controls>
  <track kind="metadata" src="/omi-list.vtt" default>
</video>
<audio id="omi-audio"></audio>
<canvas id="omi-canvas"></canvas>
...

```

§ 45. The Pipeline in Code

```

...ts
const audioCtx = new AudioContext();
const panner = audioCtx.createPanner();

const track = video.textTracks[0];
track.addEventListener('cuechange', () => {
  const cue = track.activeCues[0];
  const payload = JSON.parse(cue.text);

  // Validate against the regex
  const range = payload.range;
  const layer = payload.layer;
  const token = payload.token;

  const constraint = getConstraintForLayer(layer);
  if (!constraint.test(token)) return;

  // Accepted: update the observer position
  const position = computePositionFromRange(range);
  panner.positionX.setValueAtTime(position.x, audioCtx.currentTime);
  panner.positionY.setValueAtTime(position.y, audioCtx.currentTime);
  panner.positionZ.setValueAtTime(position.z, audioCtx.currentTime);

  // Update the DOM geometry
  const quad = computeQuadFromRange(range);
  const hitArea = document.getElementById('hit-range-${range}');
  hitArea.coords = quadToCoords(quad);

  // Update the trace log
  traceLog.append({ range, layer, token, position, quad });
});
...

```

The pipeline is HTTP → TextTrack → regex → PannerNode → DOM geometry → media.

Part XV — The Canonical Statement

§ 46. The Full Model

```

...
Primitive:  Atomics.compareExchange
Phases:    bind, apply, eval

```

Closure: digest (read, consider, print)
Base: iff (position $\Leftrightarrow$ period)
Core type: symmetric knot
Ruler: 8 slots (2! + 3!)
Pipeline: -4D to -1D range selector (0x00..0x3F)
Authorities: OMI, Tetragrammatron, Metatron, IMO
Carrier: HTTP/1.1 + TextTracks + MediaStreams
Validation: regex + Horn clauses
Projection: DOM geometry
...

§ 47. The Lisp

OMI-Lisp is:

...

A Lisp that emerges from a single primitive
where code is data
where the program is a swap list
where the evaluation is structural
where the notation is translatable
where the authorities are standard web primitives
...

A Lisp that runs natively on the web.

§ 48. The Emergence

The Lisp emerges from the primitive because:

- Every Lisp form is a composition of the primitive's phases
- Every Lisp property is a consequence of the primitive's structure
- Every Lisp operation reduces to bind, apply, eval, or digest

The primitive is Lisp-shaped. The Lisp is the primitive's elaboration.

Part XVI — The Final Statement

OMI-Lisp is an Atomic Compare-and-Exchange Lisp whose primitive is `Atomics.compareExchange`, whose core type is the symmetric knot, whose operations are bind, apply, eval, digest, whose base is the iff, whose ruler is 8 slots (2! + 3!), whose pipeline is the -4D to -1D range selector over 0x00..0x3F, and whose authorities are OMI, Tetragrammatron, Metatron, and IMO.

The notation is translatable to standard web primitives:

- HTTP/1.1 headers replace `omi---imo`
- WebVTT cues replace `o---o/--/?---@---@`
- `<dt>/<dd>` pairs replace (a . b)
- `data-*` attributes replace the one-long-word
- DOM description lists replace the receipt ring

The pipeline is preserved. The authorities are preserved. The laws are preserved.

Only the notation changes. The Lisp remains.

The primitive is the Lisp. The Lisp is the primitive's elaboration.

End of OMI-Lisp: Emergence from `Atomics.compareExchange`.